SRD: A Multi-Tenant Architecture for Digital Management of Community Action Boards

Juan Sebastián González Horta
Servicio Nacional de Aprendizaje (SENA) · Neiva, Colombia

Abstract— Community Action Boards need to manage membership records, documents, meetings, and communications while preserving data separation across organizations. This applied, descriptive study examines the SRD multi-tenant architecture and its documented technical results through 19 October 2026, complemented by a repository review and a contextual instrument dated 3 November. SRD includes a Laravel API Gateway, ten domain services, a separate MySQL database per service, and a React web interface; Android and Windows clients remain partial. The original cutoff documented 33 passing browser test files among 140 available on a single deployed web image; this is not an overall acceptance rate. Two administrative scenarios, a problem tree, and a twelve-source technology surveillance matrix are incorporated. A contextual form records three tasks completed without assistance, two with assistance, and ratings of 4/5; these entries do not amount to an independent evaluation with users from a real community board. The study reports reproducible technical progress and outstanding validation limitations.

Keywords— multi-tenancy, community organizations, microservices, auditing, security, applied research

I. INTRODUCTION

Colombian community action organizations have a distinct institutional role, addressed by Law 2166 of 2021 [7]. Their daily administration requires maintaining membership records, scheduling activities, storing documents, tracking resources, and identifying who changed information. When these activities depend on disconnected files and applications, administrators face difficulties reconstructing versions, enforcing permissions, and maintaining continuity. SRD approaches this operational challenge as a service-oriented software project; it does not replace the legal framework governing community organizations.

The research question guiding this increment was: how can a shared digital platform be built for multiple Community Action Boards while preserving organizational data separation and auditable changes through independently managed modules? The design assumption was that an API Gateway resolving authenticated sessions, compulsory organization context in each domain, and deferred publication of audit events can support these constraints as the platform grows.

The general objective was to design and technically validate an increment of SRD for the administration of multiple community boards. Specific objectives were to define the problem and business rules; organize clients, services and databases into explicit boundaries; implement representative web flows and partial companion clients; and collect test evidence and incident records so that implementation progress would not be confused with final acceptance.

II. RELATED WORK AND TECHNOLOGY SURVEILLANCE

Multi-tenant architecture permits multiple organizations to use shared application infrastructure, but introduces explicit trade-offs in data partitioning, customization and recovery. Tsai et al. [1] discuss such SaaS concerns; Krebs, Momm and Kounev [2] distinguish tenancy from the mere sharing of computing resources. This distinction matters for SRD: its MySQL databases are partitioned by functional domain, not by individual community board. Organization-level context separates records and operations within each domain.

Security does not follow automatically from placing an access check at the entry point. Pervez et al. [3] explore secure multi-tenant SaaS architecture; Singh et al. [4] address protections among tenants. Krebs and Loesch [5] show that performance isolation and data isolation require different reasoning, while Reinhardt et al. [6] discuss provisioning choices in mixed-tenancy environments. Collectively, these contributions motivate evaluating authorization, data ownership, operational recovery, and concurrent performance as distinct engineering questions.

The technical review also used authoritative guidance. Colombian Law 2166 [7] establishes the relevant organizational setting; OWASP ASVS 5.0 [8] provides a security verification framework for web applications; and NIST SP 800-63B-4 [9] addresses authentication and authenticator management. SRD draws on these resources when defining testable controls, including backend authorization, session handling and an additional verification step. The project is not claimed to be certified against these standards.

Comparable products include CiviCRM [10], an open-source constituent-management system for non-profits, and enterprise platforms such as Odoo [11] and ERPNext [12], which support inventory and operational workflows. These examples demonstrate existing solution patterns, not a feature-by-feature equivalence. The targeted gap for SRD is narrower: the coordination of people, events, internal records and informative treasury tasks under community-board-specific roles. No claim of superior breadth or commercial performance is made.

The twelve-reference technology surveillance matrix is included in Appendix A. Selection criteria were relevance to multi-tenant isolation, security and authentication, legal context for Colombian community boards, and functional comparators. The search was exploratory rather than systematic. Items [1]–[6] are technical studies; [7] is legislation; [8]–[9] are security standards/guidelines; [10]–[12] are product descriptions, not peer-reviewed papers. Each source is linked to a specific SRD design decision or limitation.

III. METHODOLOGY

PROBLEM DEFINITION AND DOCUMENTED CASES

The problem was delimited using two administrative scenarios described in instrument E-JAC-01, dated 3 November 2026 and contextualized around the Guasimal Community Action Board (Palermo, Huila, Colombia). CASE-01: membership information is kept across

spreadsheets and paper records, which may hinder retrieval and updates. CASE-02: meeting minutes are stored in separate folders and computers, making it difficult to identify the latest version. Both scenarios are qualitative design inputs; they are not presented as independent measurements from a real organization.

The problem tree was structured as follows: direct causes: dispersed membership records, documents without a single controlled repository, and notices circulated through informal messaging; central problem: insufficient traceability and timely retrieval in the board's administrative processes; potential effects: slower searches, inconsistent versions, missed announcements, and reliance on individual officeholders to reconstruct decisions. These links are analytical assumptions, not statistically established causal relationships.

EFFECTS: slow retrieval; document duplication; missed notices; difficulty reconstructing decisions.
↑
CENTRAL PROBLEM: insufficient traceability and timely access in board administrative processes.
↑
CAUSES: dispersed paper/digital membership lists; minutes with no version control; informal messaging.

Fig. 1. Problem tree derived from the administrative scenarios under review.

The proposed solution addresses these causes using People, Files, Calendar, Notifications and Audit modules with organization-scoped access control. The problem-to-feature mapping produces future evaluation criteria: retrieving a past meeting minute, viewing an up-to-date membership record, identifying the origin of a change, and finding an announcement without searching multiple separate channels.

FIELD INSTRUMENT AND VISUAL WALKTHROUGH

Instrument E-JAC-01 was reviewed. It is dated 3 November 2026 at 19:00 and contains participant code P01 and ten questions about membership, minutes, documents, treasury, inventory, and communications. The form attributes the answers to the chair of the Guasimal board and has boxes marked for verbal consent. Described needs include retrieval of dispersed records, identification of current document versions, delivery of announcements, and mobile access. The attributed participant's name, identification number, and other personal details are intentionally omitted.

The record is treated as a contextual instrument with an exploratory scope. Although the current version contains completed answers and marked consent, the supplied documentation alone does not establish that an independent interview with a real community-board representative took place. Its descriptions are therefore used to define needs and design criteria, not to infer representativeness, generalize findings to real users, or replace field verification.

The form also contains a visual review of the initial SRD demonstration dashboard in Chrome, with a displayed timestamp of 23:25 on 3 November 2026. Five tasks are assigned C/C/C/A/A (C: completed unassisted; A: completed with assistance), usefulness and ease-of-use scores of 4/5, and successful access entries for the superadministrator and administrator roles. These are recorded entries in the form; the only supplied screenshot depicts the main dashboard and does not show each navigation action. The entries are not treated as a reproduced functional test or an independent usability evaluation with external users.

The historical technical observation period ended on 19 October 2026. A supplementary documentary review on 4 November assessed the repositories and the E-JAC-01 instrument without retroactively changing original execution logs. Three levels of evidence are distinguished: (a) technical tests and checks documented at the original cutoff, (b) subsequent QA results reported in the repositories but not rerun during this review, and (c) responses and task outcomes listed in the contextual form without verified external administration.

This project used an applied, descriptive technology case-study approach. The unit of analysis was the SRD 0.1 increment and the technical evidence compiled through 19 October 2026. Primary materials included requirements, source code, database migrations, OpenAPI contracts, Docker Compose configuration, automated tests, and operational recovery logs. Relevant literature was examined around multi-tenancy, security, digital identity, and comparable platforms.

The work followed five phases. First, a baseline comprising 56 functional requirements, 18 non-functional requirements and 20 use cases was established for community-board activities. Second, service boundaries and a gateway were selected. Third, representative local flows were implemented and exercised with synthetic test data. Fourth, records from unit, browser, and operational checks were classified by evidence type. Fifth, corrective diagnosis was carried out on migration and audit-delivery failures in a Docker development environment.

The technical instruments comprised requirements traceability, test cases, PHP/Jest/Playwright results, Docker health checks and transactional outbox counters. Form E-JAC-01 was received as supplementary documentation on 3 November 2026; its provenance as a field interview and the pilot task marks require verification before being treated as empirical end-user validation.

To avoid overstatement, each claim was classified as an automated test documented in the repository, a manual operational observation, or an outstanding feature. Healthy containers were not treated as proof that every business requirement was accepted. Likewise, the project did not present an integrated concurrency benchmark for the complete platform.

A practical consistency matrix linked each research objective to a means of verification. Organization-level separation was mapped to context validation in the Gateway and domains; traceability was mapped to event contracts and outbox records; and maintainability was mapped to migration, test and backup scripts. This mapping ensures that a technical statement can be traced back to a file and a reproducible check. It does not, however, substitute for an independent review of the complete system.

Case selection emphasized higher-risk flows: membership authentication, organization-scoped queries, concurrent folder changes, informative treasury movements, inventory

histories, and audit delivery. Demonstration fixtures were deliberately synthetic to avoid using real residents, private email messages, or financial transactions during local experiments. The review retained only test outcomes explicitly supported by repository validation notes or by operational logs generated during the documented recovery session.

The literature search used terms including multi-tenancy AND security, multi-tenant SaaS AND architecture, community organization information systems, and equivalent Spanish phrases. Papers [1]–[6] were selected for their treatment of resource isolation, protection and architecture; references [7]–[9] for legal and technical guidance; and [10]–[12] as examples of existing solutions. This is a focused exploratory review, not an exhaustive systematic review or an empirical feature benchmark across products.

IV. SYSTEM DESIGN AND IMPLEMENTATION

The web interface is built with React 19, TypeScript and Vite. Laravel powers the entry Gateway and domain applications; MySQL 8.4 provides per-domain databases. Ten domains are present: identity, configuration, records, audit, files, calendar, notifications, treasury, inventory and chat. A separate specialized reporting service remains within the planned architecture, not the implemented code. The Electron Windows client uses the web application hosted on the server, while the native Android client and an Expo Go variant cover only selected workflows.

The Gateway validates a session and forwards a signed internal context containing user, organization, and permissions. Each domain revalidates access to its own resources, so hiding interface controls does not substitute for server authorization. Databases and migrations belong to their respective domains. Although several boards share service infrastructure, record lookups must remain restricted to the verified active organization and cannot rely on a tenant identifier freely supplied by a client.

Auditing uses a transactional outbox. The originating service stores an event together with its business change, and a scheduler attempts delivery to the audit service. Stable event identifiers allow the receiver to avoid duplicate stored audit rows after retries. Eventual delivery introduces a visible operational state: an event may be ready, deferred, delivered or exhausted. SRD exposes counters to help administrators identify these conditions rather than assuming every event reaches its target immediately.

The Files domain uses private storage, upload constraints and ClamAV scanning for applicable content. Treasury maintains informative COP entries and corrective reversals rather than editing confirmed records. Inventory maintains asset histories through movements. Such decisions deliberately place consistency and traceability mechanisms in backend rules and database operations, not merely in forms displayed by a browser.

V. RESULTS

At the 19 October cutoff, 33 distinct browser test files were documented as passing among 140 available files on the same Docker-served web image. No final result was recorded for the other 107 on that image; therefore, 33/140 is neither a whole-suite pass rate nor a project-completion measure. According to the project validation notes, an empty-state browser run covered fourteen sections at 320- and 1280-pixel widths and produced 28 screenshots. Full regression, role-based write authorization and end-to-end attachment scanning still require further checks.

In a supplementary read of the dev repositories, the Infrastructure README reports QA outcomes dated 23 October 2026: 305 of 305 PHPUnit tests, 6,292 assertions, 72 migrations checked, 11 of 11 MySQL users isolated, and 110 of 110 cross-database read attempts denied. These values are project-reported documentary evidence, not independently rerun measurements by this study. They address scoped technical controls and do not establish complete functional isolation between boards or end-user acceptance.

The examined repository contains the API Gateway, ten operational domain implementations, a React web application and partial desktop and Android clients. Its requirements traceability records 52 functional requirements as partial and four as pending; those categories must not be converted into a completion percentage. The recorded Files unit tests passed 32 cases with 778 assertions. Separate Chromium runs passed a bilingual folder test and a hierarchy/conflict regression. Android development evidence includes 37 successful Jest tests and a built x86_64 debug package.

On October 9, the audit service database was initially missing its audit-event table, which caused sender outboxes to exhaust delivery attempts. After applying the missing migration and backing up affected databases, 15 Identity events, eight Records events, four Calendar events, three Treasury events, and three Inventory events were recovered. All 33 were subsequently reported as published with zero exhausted in those emitters. Files and Chat migrations were applied, and their scheduler containers eventually transitioned to healthy.

A second incident concerned the signature updater. ClamAV itself was healthy, but the FreshClam script failed because shell input contained Windows CRLF line endings. Converting the script to LF and recreating only the updater enabled the signature database test and upgraded daily.cld to version 28148. The marker file was visible from the Files service. This is evidence of signature update and availability, not a completed end-to-end malicious-file rejection test.

The technical demonstration included eight pending member records, four calendar events, three informative treasury entries, and three inventory assets, all synthetic. An earlier physical Android test documented login, email MFA, Home, People, Calendar, Alerts, Profile, and session persistence with Expo Go. Instrument E-JAC-01 additionally records three tasks marked C, two marked A, usefulness and ease-of-use scores of 4/5, and a request for a short getting-started guide. These entries describe the form and do not replace an observed user study with complete interaction records.

Regarding authorization, technical materials show that the Gateway resolves the session and signs the context sent to each domain. This leads to a testable implication: a hidden button is not an access-control mechanism in itself; backend

tests must verify operations for each role. The examined project includes a role matrix covering SA, AD, RE, TE, AU and CO as well as delegated folder-reading permissions. Such rules require regression testing after membership changes, session revocations, and organization suspension.

Regarding persistence, domain-specific OpenAPI contracts and migrations are present in the repository. The Files and Chat incident illustrates why those two artifacts must be checked together: an HTTP process could be available while the corresponding database still had no tables. After applying migrations, outbox queries no longer reported the missing table and the two scheduler containers reached healthy status. Although narrowly scoped, this observation provides reproducible evidence that an infrastructure availability signal does not prove functional readiness.

The findings should be interpreted at three separate levels: infrastructure deployment, component-level test results, and acceptance of actual community-board administrative workflows. Docker health checks and migrations address the first; automated suites and browser flows address the second; the third remains open. Maintaining these distinctions makes it possible to report real engineering progress without inventing completion percentages for the 56 requirements or extrapolating local evidence to a public deployment.

VI. DISCUSSION AND LIMITATIONS

Instrument E-JAC-01 offers administrative scenarios relevant to SRD modules and records a demonstration walkthrough, but does not independently corroborate an interview or provide a full trace of five user interactions. Future evaluation should include verifiable consent, effective participation by a real community-board member, task-by-task recording, and observation of errors. Even a verified one-person sample would not establish adoption across all community organizations.

These results illustrate both the benefit and operational cost of modular design. The missing audit table affected event delivery but could be diagnosed without deleting business records. The signature updater was repaired without reinstalling the database. At the same time, each independent service carries separate migration, configuration, health-check, and integration responsibilities. Creating architectural boundaries therefore improves fault localization but does not remove the work of operating them.

The outbox provides stronger traceability under transient failures, although exhausted events still require an authorized recovery procedure and monitoring. A healthy malware scanner does not replace file-type controls, quotas or access checks. Similarly, maintaining many organizations in one application requires repeatable cross-tenant authorization tests; it would be unwarranted to infer complete isolation from a single successful demonstration.

The main limitations are the absence of a confirmed external interview and an independently verifiable pilot with real community-board users, incomplete requirement acceptance, pending full browser regression, an unimplemented specialized Reports service, and partial Chat, Android, and Windows capabilities. Production-grade TLS/TURN, operational continuity, and signed installers have not been demonstrated. This paper therefore reports a technological increment rather than certifying a production-ready system.

VII. CONCLUSIONS AND FUTURE WORK

Future work includes obtaining verifiable evidence from a real Community Action Board through a semi-structured interview and an observed pilot, safeguarding consent records, logging tasks and timings consistently, and analyzing usability difficulties. Cross-board permission tests and the complete browser regression must also be performed on a fixed release. New findings should be reported in a separate observation period.

SRD demonstrates a technically verifiable foundation for a shared digital platform for community organizations, using explicit organization context, per-domain databases and observable audit delivery. Automated checks, Docker diagnostics and narrowly scoped corrective actions made it possible to validate specific engineering mechanisms while retaining a realistic record of limitations.

The next stage should complete outstanding requirements, execute systematic cross-organization access tests, validate clean-file and EICAR scanning, measure concurrent load, apply instruments to actual users and prepare a secure production deployment. The central methodological lesson is that developing a feature and demonstrating that it satisfies an acceptance criterion are different activities. Reliable claims require identifiable tests, synthetic or consented datasets, dates and reproducible evidence.

REFERENCES

[1] W.-T. Tsai, Q. Shao, Y. Huang, and X. Bai, “Towards a scalable and robust multi-tenancy SaaS,” 2010. doi:10.1145/2020723.2020731.

[2] R. Krebs, C. Momm, and S. Kounev, “Architectural Concerns in Multi-Tenant SaaS Applications,” 2012. https://www.scitepress.org/PublishedPapers/2012/39576/pdf/index.html

[3] Z. Pervez, S. Lee, and Y.-K. Lee, “Multi-Tenant, Secure, Load Disseminated SaaS Architecture,” ICACT, 2010. https://www.icact.org/upload/2010/0265/20100265_finalpaper.pdf

[4] A. Singh, S. Kumar J., and A. Muralidhar, “A Secure Multi-Tenant Model for SaaS System,” International Journal of Computer Applications, vol. 90, no. 15, 2014. doi:10.5120/15793-4468.

[5] R. Krebs and M. Loesch, “Comparison of Request Admission Based Performance Isolation Approaches in Multi-tenant SaaS Applications,” 2014. https://www.scitepress.org/PublishedPapers/2014/49488/pdf/index.html

[6] M. Reinhardt, S. T. Ruehl, S. A. W. Verclas, U. Andelfinger, and A. Schütte, “Architectural Design of a Deployment Platform to Provision Mixed-tenancy SaaS-Applications,” 2014. https://www.scitepress.org/PublishedPapers/2014/48138/pdf/index.html

[7] Congress of Colombia, “Law 2166 of 2021: community action organizations,” 2021. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=184758

[8] OWASP Foundation, “Application Security Verification Standard (ASVS),” version 5.0.0, 2025. https://github.com/OWASP/ASVS

[9] NIST, “Digital Identity Guidelines: Authentication and Authenticator Management,” SP 800-63B-4, 2025. doi:10.6028/NIST.SP.800-63B-4.
[10] CiviCRM, “CiviCRM Core: constituent relationship management for non-profit organizations,” https://github.com/civicrm/civicrm-core
[11] Odoo, “Inventory: Features,” https://www.odoo.com/app/inventory-features
[12] Frappe Technologies, “ERPNext,” https://erpnext.com
[13] J. S. González Horta, “SRD: README, VALIDACION, TRAZABILIDAD, ARQUITECTURA and source code,” SRD repositories, technical cutoff of 19 October 2026 and supplementary review on 4 November 2026. https://github.com/jsgh1/SRD-Docs

DISCLOSURE OF TOOLS USED

OpenAI ChatGPT (GPT-6) was used during the bilingual revision of 4 November 2026 to support drafting, translation, and organization of the source matrix. Technical evidence was checked against the SRD repositories, while E-JAC-01 data retained its documentary status. No real interviews or experimental results were generated by this tool. Editable files were prepared using python-docx and PDFs using LibreOffice. The author remains academically responsible for the article and its oral defense.

APPENDIX A. TECHNOLOGY SURVEILLANCE MATRIX

Ref.	Critical synthesis and relevance to SRD
[1]	Tsai et al. (2010): SaaS scalability and robustness under multi-tenancy; informs logical separation and capacity questions. Does not test SRD.
[2]	Krebs et al. (2012): architectural concerns in multi-tenant systems; separates tenancy from general resource sharing. Domain boundaries differ from organization boundaries.
[3]	Pervez et al. (2010): secure multi-tenant SaaS and load distribution; motivates cross-board authorization tests and gateway controls.
[4]	Singh et al. (2014): security models for multi-tenant SaaS; supports server-side policies rather than merely hidden UI controls.
[5]	Krebs and Loesch (2014): performance isolation strategies; motivates separate tests for load isolation and data isolation.
[6]	Reinhardt et al. (2014): mixed-tenancy deployment architecture; frames provisioning options without proving benefits in this prototype.
[7]	Colombian Law 2166 of 2021: organizational context for community action bodies; informs actors and workflows, not interface validation.
[8]	OWASP ASVS 5.0.0 (2025): verifiable web application security requirements; informs authorization and session criteria, not certification.
[9]	NIST SP 800-63B-4 (2025): authentication and authenticator guidance; informs session/MFA checks, without proving implementation.
[10]	CiviCRM: open-source constituent relationship management for nonprofits; useful functional comparison, without verified Colombian legal equivalence.
[11]	Odoo Inventory: reference product for stock traceability; exploratory inventory comparison, not a benchmark.
[12]	ERPNext: open modular ERP product; exploratory scope comparison, not evidence of SRD superiority.

The matrix includes six SaaS architecture and security studies [1]–[6], one Colombian law [7], two technical guidelines [8]–[9], and three functional reference platforms [10]–[12]. Each synthesis links the source to a design or evaluation decision in SRD.